Database Management & Warehousing

Revision Sheet

Every topic on its own page • rules, method, worked example, traps

How to use this sheet

- **One topic per page.** Nothing splits across pages — read a page, close the book, and try to reproduce the formula box from memory.
- **Read in this order on each page:** the italic line (what the topic *is*) → the formula box → "how to approach" → the worked example → the traps.
- **The dashed "My notes" box is yours.** Add the variant that caught you out, a formula your coaching notes state differently, or the page number of a question you got wrong.
- **Traps in the orange box are the marks people actually lose.** Re-read only those the night before the exam.

Contents

1. ER Model & the Relational Model
2. Relational Algebra & Tuple Calculus
3. SQL
4. Integrity Constraints & Normal Forms
5. Functional Dependencies & Keys
6. File Organization & B / B+ Tree Indexing
7. Data Transformation & Preprocessing
8. Data Warehousing & OLAP

1. ER Model & the Relational Model

The ER diagram is the design; the relational schema is the implementation. Questions ask you to convert between them and count tables.

KEY FORMULAS

Vocabulary

$$\text{degree} = \#\text{ columns}, \qquad \text{cardinality} = \#\text{ rows}$$

WHAT YOU MUST KNOW

- A **weak entity** has no key of its own; it borrows the owner's key.
- A **multivalued attribute** becomes its own relation, keyed by (owner key + attribute).
- A **many-to-many** relationship becomes its own relation holding both entities' primary keys. One-to-many needs no extra table (put the FK on the many side).
- A relation is a **set** of tuples — no duplicates, no inherent order. `NULL` means unknown, not zero.

HOW TO APPROACH A QUESTION

1. Count one relation per entity, plus one per multivalued attribute, plus one per M:N relationship.
2. Check whether the question wants 3NF — that is usually the same count.
3. Total participation and cardinality constraints change *where the FK goes*, not usually the table count.

WORKED EXAMPLE

Q. $E_1(A_{11}, A_{12}, A_{13})$, $E_2(A_{21}, A_{22}, A_{23})$ with A_{22} multivalued; R_{12} is M:N with total participation. Minimum 3NF relations?

One for E_1; one for E_2 (without A_{22}); one for the multivalued A_{22}; one for the M:N relationship:

$$1 + 1 + 1 + 1 = \mathbf{4}$$

WATCH OUT

- Keeping a multivalued attribute as a column — that breaks 1NF.
- Creating a separate table for a 1:N relationship (a foreign key is enough).

MY NOTES

2. Relational Algebra & Tuple Calculus

Algebra says how to compute the answer; calculus says what the answer is. GATE asks you to read both.

KEY FORMULAS

Core operators

$$\sigma \text{ (select rows)}, \quad \pi \text{ (project columns)}, \quad \rho \text{ (rename)}$$

Sizes

$$|R \times S| = |R| \cdot |S|, \qquad |R \bowtie S| \leq |R| \cdot |S|$$

WHAT YOU MUST KNOW

- π **removes duplicates** (a relation is a set) — unlike SQL's `SELECT`.
- Primitive complete set: $\sigma, \pi, \times, \cup, -, \rho$. Join and intersection are derived.
- $\cup, \cap, -$ need **union compatibility**: same number of attributes with matching domains.
- Natural join keeps only tuples agreeing on *all* shared attributes; division answers "related to *all* of S".

HOW TO APPROACH A QUESTION

1. For a calculus expression, read it as: "all tuples t such that ... exists / for all ...".
2. Evaluate condition by condition against the given rows — these are small tables by design.
3. Count the surviving tuples; remember duplicates vanish under projection.

WORKED EXAMPLE

Q. $X = \{(P1, Q1, R1), (P2, Q2, R2), (P3, Q3, R2)\}$, $Y = \{(P1, 10), (P1, 15), (P2, 20), (P3, 1)\}$.
Query: tuples of X whose P has some Y row with $S > 1$. $P1$: 10,15 qualify. $P2$: 20 qualifies. $P3$: only 1, fails.

$$\text{answer} = \mathbf{2} \text{ tuples}$$

WATCH OUT

- Forgetting that projection eliminates duplicate rows.
- Applying $\cup$ or $-$ to relations that are not union compatible.

MY NOTES

3. SQL

Know the evaluation order and the NULL rules; most SQL questions are decided by those two things alone.

KEY FORMULAS

Logical evaluation order

`FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY`

WHAT YOU MUST KNOW

- `WHERE` filters *rows* before grouping; `HAVING` filters *groups* after aggregation.
- `SELECT` **keeps duplicates** unless you write `DISTINCT`; `UNION` removes them, `UNION ALL` keeps them.
- `COUNT(*)` counts all rows; `COUNT(col)` skips `NULL`s. Any comparison with `NULL` is `UNKNOWN` — use `IS NULL`.
- A **correlated** subquery references the outer row, so it is conceptually re-evaluated for every outer row.

HOW TO APPROACH A QUESTION

1. For a correlated subquery, take the outer rows **one at a time** and evaluate the inner query for each.
2. Tabulate: one row of working per outer row, then count the rows that satisfy the condition.
3. Watch strict versus non-strict comparisons ($<$ vs $\leq$) — ties are usually the point.

WORKED EXAMPLE

Q. `Account` balances $5000, 5000, 10000, 15000, 18000$. Return `AccNo` where (# with a strictly larger balance) $\geq$ (# with a strictly smaller balance).

5000: larger $= 3$, smaller $= 0$ **yes** (twice). 10000: $2 \geq 2$ **yes**. 15000: $1 \geq 3$ **no**. 18000: $0 \geq 4$ **no**.

rows returned $=$ **3**

WATCH OUT

- Counting equal values as "larger" or "smaller" when the comparison is strict.
- Putting an aggregate in `WHERE` instead of `HAVING`.

MY NOTES

4. Integrity Constraints & Normal Forms

Each normal form removes one specific kind of redundancy. Name the highest form a relation satisfies, given its FDs.

KEY FORMULAS

The ladder

$$\text{BCNF} \Rightarrow \text{3NF} \Rightarrow \text{2NF} \Rightarrow \text{1NF}$$

3NF condition

$$\text{for each } X \to Y: \ X \text{ is a superkey } \textbf{or } Y \text{ is prime}$$

BCNF condition

$$\text{for each non-trivial } X \to Y: \ X \text{ is a superkey}$$

WHAT YOU MUST KNOW

- 1NF: atomic values. 2NF: no **partial** dependency on part of a key. 3NF: no **transitive** dependency.
- A **prime attribute** belongs to some candidate key — that word is what separates 3NF from BCNF.
- 3NF decomposition is always lossless **and** dependency-preserving; BCNF is lossless but may *lose* dependencies.
- Referential integrity: an FK value must match an existing PK or be `NULL`. Entity integrity: a PK is never `NULL`.

HOW TO APPROACH A QUESTION

1. Find the candidate keys first — you cannot classify anything without them.
2. For each FD, ask: is the left side a superkey? If not, is the right side prime?
3. All FDs pass the superkey test ⇒ BCNF; otherwise check the 3NF relaxation.

WORKED EXAMPLE

Q. Relations $r(A, B)$ and $s(E, C)$; A is the PK of R, E is an FK of S referencing A. Which operations can *never* violate the FK constraint?

Inserting into r only adds referenceable values; deleting from s only removes references. Deleting from r can orphan a reference; inserting into s can add an unmatched one.

insert into r and **delete from** s

WATCH OUT

- Testing normal forms before finding the candidate keys.
- Forgetting the "or Y is prime" escape clause that makes 3NF weaker than BCNF.

MY NOTES

5. Functional Dependencies & Keys

Attribute closure is the single algorithm here. Everything — keys, superkeys, normal forms — comes from it.

KEY FORMULAS

Closure

$$X^+ : \text{ repeatedly add the RHS of any FD whose LHS} \subseteq X$$

Key test

$$X \text{ is a superkey} \iff X^+ = \text{all attributes}$$

Counting superkeys

$$\text{single-attribute key over } n \text{ attrs} \Rightarrow 2^{n-1} \text{ superkeys}$$

WHAT YOU MUST KNOW

- A **candidate key** is a *minimal* superkey — no proper subset is a superkey.
- **An attribute that never appears on any right-hand side must be in every candidate key.** Start there, always.
- Armstrong's axioms: reflexivity, augmentation, transitivity (sound and complete).
- An FD $X \to Y$ is trivial when $Y \subseteq X$.

HOW TO APPROACH A QUESTION

1. List the attributes that appear *only* on left sides — they seed every candidate key.
2. Take that seed's closure; if it is everything, it is the unique key.
3. Otherwise add one attribute at a time and re-close, keeping only minimal sets.

WORKED EXAMPLE

Q. $R(A, B, C, D, E)$ with $F = \{A \to BC,\ CD \to E,\ E \to A\}$. Find the candidate keys.
D never appears on a right side, so D is in every key. $\{A, D\}^+$: $A \to BC$ gives $ABCD$, then $CD \to E$ gives everything. Likewise $\{E, D\}$ (via $E \to A$) and $\{C, D\}$ (via $CD \to E \to A$).

AD, **ED**, **CD**

WATCH OUT

- Stopping at the first key found — there are usually several.
- Calling a superkey a candidate key without checking minimality.

MY NOTES

6. File Organization & B / B+ Tree Indexing

Indexes trade space for speed. The classic NAT question is a fan-out calculation from byte sizes.

KEY FORMULAS

Non-leaf node capacity

$$n \cdot P + (n-1) \cdot K \ \leq \ \text{node size}$$

Search cost

$$O(\log_f N) \quad (f = \text{fan-out})$$

WHAT YOU MUST KNOW

- In a **B+ tree**, data pointers live *only* in the leaves, and the leaves are linked — which is why range queries are fast. A B tree stores data in internal nodes too.
- A non-leaf node with n node pointers holds $n-1$ keys.
- **Dense** index: one entry per record. **Sparse**: one per block. A secondary index (non-ordering field) *must* be dense.
- Hash index: great for equality, useless for ranges. B+ tree: good for both.

HOW TO APPROACH A QUESTION

1. Write the inequality $nP + (n-1)K \leq$ node size, solve for n, then **floor** it.
2. Data-record pointer size is a distractor for *non-leaf* questions — non-leaves have none.
3. For a leaf node, use key + data-record pointer per entry, plus the sibling pointer.

WORKED EXAMPLE

Q. Node = 4096 B, node pointer = 10 B, key = 11 B. Max node pointers in a non-leaf?

$$10n + 11(n-1) \leq 4096 \ \Rightarrow \ 21n \leq 4107 \ \Rightarrow \ n \leq 195.57$$

$$n = \mathbf{195}$$

WATCH OUT

- Including the data-record pointer size in a non-leaf calculation.
- Rounding up instead of flooring.

MY NOTES

7. Data Transformation & Preprocessing

Shared with the Machine Learning paper — learn these formulas once and collect marks in two subjects.

KEY FORMULAS

Min–max normalisation

$$x' = \frac{x - \min}{\max - \min} \in [0, 1]$$

z-score standardisation

$$z = \frac{x - \mu}{\sigma} \quad \text{(mean 0, sd 1)}$$

WHAT YOU MUST KNOW

- **Discretization** converts a continuous attribute into bins: *equal-width* (same numeric range each) vs *equal-frequency* (same number of points each).
- **Sampling** takes a representative subset; *stratified* sampling draws proportionally from each group.
- **Lossless** compression reconstructs the original exactly; lossy discards information.
- Min–max is bounded but outlier-sensitive; z-score is unbounded but handles outliers better.

HOW TO APPROACH A QUESTION

1. Identify which normalisation the question names — the two give very different numbers.
2. Plug in carefully; these are one-line NAT or MCQ questions.
3. For binning, sort the data first, then split by width or by count as instructed.

WORKED EXAMPLE

Q. Income has min = 46000, max = 170000, $\mu = 96000$, $\sigma = 21000$. z-score of 106000?

$$z = \frac{106000 - 96000}{21000} = \frac{10000}{21000} \approx \mathbf{0.476}$$

Note the min and max are decoys — z-score never uses them.

WATCH OUT

- Using min/max inside a z-score (or μ, σ inside min–max) — the question supplies all four on purpose.
- Confusing equal-width with equal-frequency binning.

MY NOTES

8. Data Warehousing & OLAP

A warehouse is built for analysis, not transactions. The five OLAP operations are pure definition recall.

KEY FORMULAS

Warehouse definition

$$\text{subject-oriented},\ \text{integrated},\ \text{time-variant},\ \text{non-volatile}$$

WHAT YOU MUST KNOW

- **Star schema**: one central fact table (numeric measures + FKs) surrounded by *denormalized* dimension tables. **Snowflake**: the dimensions are normalized into sub-tables.
- **Roll-up** aggregates up a concept hierarchy (Month → Year); **drill-down** goes to finer detail (Country → State).
- **Slice** fixes one dimension (cube loses a dimension); **dice** selects ranges on several; **pivot** rotates the view.
- An **additive** measure (`SUM(sales)`) aggregates over every dimension; an average is non-additive.

HOW TO APPROACH A QUESTION

1. Ask whether the level of detail became *finer* (drill-down) or *coarser* (roll-up).
2. If the number of dimensions drops to a fixed value, it is a slice; if several are restricted, a dice.
3. OLTP = many small writes; OLAP = few large analytical reads.

WORKED EXAMPLE

Q. A cube shows Sales by (Product Type, Month, Country). You now want it by (Product Type, Month, *State*). Which OLAP operation?

State is *below* Country in the location hierarchy — you moved to finer detail:

Drill-down

WATCH OUT

- Calling drill-down "roll-up" — check which way the hierarchy moves.
- Confusing slice (one dimension fixed) with dice (several restricted).

MY NOTES